# Artefactos de Código — Sistema de Lavado Vehicular

## Portada

**Tabla 1**

*Datos de identificación del documento*

| Campo | Valor |
|---|---|
| Proyecto | Sistema de Gestión de Lavado Vehicular (Vehicle Washing / LavaRápido) |
| Producto | Artefactos de código (sección 2.5.4) |
| Programa | Tecnólogo en Análisis y Desarrollo de Software (ADSO) |
| Ficha | 3145556 |
| Aprendices | Brayan Estiven Patiño Cabrera, Juan Camilo Guilombo Osorio, Merari Urbano Quintero |
| Instructora | Karol Daniela Correa |
| Centro de formación | Industria |
| Fecha | 21 de octubre de 2026 |

El código del sistema está en 10 repositorios de GitHub: la aplicación web, la aplicación móvil y 8 del backend. Cada uno tiene un flujo de GitHub Actions (`.github/workflows/ci.yml`) que compila y corre sus pruebas en cada cambio (en `lavarapido-infra` valida los scripts de despliegue, los archivos de Docker Compose y la sintaxis de las pruebas de punta a punta); `lavarapido-infra` tiene además `deploy.yml`, un flujo que se lanza a mano para actualizar el servidor. Este documento inventaría los dos clientes y explica cómo compilarlos y ejecutarlos; el backend se describe solo en lo necesario para que los clientes funcionen. No se reproduce el código fuente.

## 1. Inventario de repositorios

**Tabla 2**

*Repositorios del sistema (rama `develop`, corte 21 de octubre de 2026)*

| Artefacto | Repositorio | Tecnología | Commits | Último commit |
|---|---|---|---|---|
| Aplicación web | Front-end-proyecto-web | Angular 21.2.25, TypeScript 5.9 | 474 | 2026-10-21 |
| Aplicación móvil | LavadoVehicular-Movil | React Native 0.86.3, Expo ~57, TypeScript 6.0 | 236 | 2026-10-21 |
| Infraestructura | lavarapido-infra | Docker Compose | 86 | 2026-10-21 |
| API Gateway | lavarapido-api-gateway | Spring Boot 4.0.8, Java 21 | 21 | 2026-10-21 |
| security-service | lavarapido-security-service | Spring Boot 4.0.8, Java 21 | 63 | 2026-10-21 |

| Artefacto | Repositorio | Tecnología | Commits | Último commit |
|---|---|---|---|---|
| customer-service | lavarapido-customer-service | Spring Boot 4.0.8, Java 21 | 22 | 2026-10-19 |
| booking-service | lavarapido-booking-service | Spring Boot 4.0.8, Java 21 | 61 | 2026-10-21 |
| operations-service | lavarapido-operation-service | Spring Boot 4.0.8, Java 21 | 50 | 2026-10-21 |
| payment-service | lavarapido-payment-service | ASP.NET Core 8 | 49 | 2026-10-21 |
| notification-service | lavarapido-notification-service | Spring Boot 4.0.8, Java 21 | 42 | 2026-10-21 |

**Tabla 3**

*Tamaño de los clientes*

| Artefacto | Archivos fuente (`.ts`, `.tsx`, `.html`, `.scss`) | Líneas | Pantallas |
|---|---|---|---|
| Web (`Web/src`) | 449 | 48 202 | 35 rutas |
| Móvil (`LavadoVehicular-Movil/src`) | 296 | 35 540 | 31 pantallas |

> *Nota.* Conteo del 21 de octubre de 2026 con `git ls-files` y `wc -l` sobre `src`; incluye pruebas y comentarios.

## 2. Aplicación web

La web es una SPA de Angular 21 (Angular, s.f.) con componentes standalone, carga diferida de un módulo por rol y traducción a cuatro idiomas (es, en, fr, pt). Todas las llamadas salen por `/api/v1`, que el servidor de desarrollo redirige al gateway en `localhost:8080`.

### 2.1 Estructura

**Tabla 4**

*Carpetas de `Web/src/app`*

| Carpeta | Contenido |
|---|---|
| `core/constants` | `API_BASE_URL = '/api/v1'` (`core/constants/api.ts:4`) |
| `core/guards` | `authGuard` (sesión y rol) y `guestGuard` (oculta login a quien ya tiene sesión) |
| `core/interceptors` | `authInterceptor`: agrega el JWT y cierra la sesión ante un 401 mientras la sesión sigue abierta (ignora un 401 tardío después de cerrar sesión) |

| Carpeta | Contenido |
|---|---|
| `core/services` | 17 servicios HTTP y de estado: `auth`, `booking-api`, `vehicles`, `operations-api`, `payments-api`, `promotions-api`, `loyalty-api`, `notifications`, `preferences`, `role-permissions-api`, `user-admin`, `user-session`, `inspection-api`, `inspection-photos`, `local-storage`, `notification-watcher`, `api`; `auth` además sigue el inicio y el cierre de sesión de las otras pestañas del mismo navegador |
| `core/models`, `core/utils` | Interfaces de los DTO y utilidades (por ejemplo traducción de errores de la API y `image-compress.ts`, que reduce la imagen del comprobante a máximo 2 MB antes de subirla) |
| `features/landing` | Página pública: navbar, hero, quiénes somos, características, servicios, descarga, contacto, preguntas frecuentes y pie |
| `features/auth` | Login, registro y recuperación de contraseña en tres pasos |
| `features/client` | 8 páginas del cliente + `viewmodels/client.viewmodel.ts` (layout) |
| `features/operator` | 8 páginas del operario |
| `features/admin` | 13 páginas del administrador + 5 stores (`reservations`, `payments`, `operators`, `schedule`, `business`) |
| `features/not-found` | Página de dirección no encontrada (ruta `**`) |
| `features/public-report` | Reporte de inspección de solo lectura (`/report/:token`) |
| `shared/components`, `shared/dialogs`, `shared/pipes` | 12 componentes reutilizables y 20 diálogos (confirmar, asignar operario, calificar, revisar pago, exportar…) |
| `assets/i18n` | `es.json`, `en.json`, `fr.json`, `pt.json` |

## 2.2 Dependencias principales

### Tabla 5

*Librerías de la web (`Web/package.json`)*

| Librería | Versión | Uso |
|---|---|---|
| `@angular/*` (core, router, forms, material, cdk) | 21.2.25 (Material y CDK 21.2.14) | Framework y componentes de interfaz |
| `@ngx-translate/core`, `http-loader` | ^17.0.0 | Traducciones |
| `chart.js` | ^4.5.1 | Gráficas del dashboard y reportes |
| `jspdf`, `jspdf-autotable` | ^4.2.1, ^5.0.8 | Exportar reportes a PDF |
| `xlsx`, `file-saver` | 0.20.3 (CDN de SheetJS), ^2.0.5 | Exportar a Excel y descargar archivos |
| `iconify-icon` | ^3.0.2 | Íconos |
| `vitest`, `jsdom` (desarrollo) | ^4.1.7, ^28.0.0 | Pruebas unitarias con `@angular/build:unit-test` |

## 2.3 Compilar y ejecutar

Requisitos: Node.js 24 o una versión compatible con Angular 21, y npm 11 (`packageManager: npm@11.6.2`). Para que las pantallas carguen datos, el backend debe estar arriba (sección 4).

```
cd Web
npm install          # instala dependencias
npm start            # ng serve en http://localhost:4200, con proxy /api ->
http://localhost:8080
npm run build        # build de producción en dist/web/browser
npm test             # pruebas unitarias (Vitest)
```

El proxy está en `Web/proxy.conf.json` y se activa en `angular.json` (`proxyConfig`). El build de producción tiene presupuestos de tamaño: falla si el bundle inicial pasa de 1 MB o si el estilo de un componente pasa de 16 kB (`angular.json`, `budgets`). En producción, `dist/web/browser` se sirve con cualquier servidor estático que redirija `/api` al gateway.

Según el SRS v7 (RNF-008), la web debe funcionar en las versiones vigentes de Chrome, Firefox y Edge y ser usable desde 360 px de ancho.

## 3. Aplicación móvil

La app móvil está hecha con Expo 57 sobre React Native 0.86 (Expo, s.f.; React Native, s.f.) y está dirigida a Android 8.0 o superior (RNF-008 del SRS v6). La versión web (`react-native-web`) se usa solo para desarrollo y pruebas. Habla solo con el gateway y descubre sola la IP del computador que corre Expo, así que no hace falta configurar nada en una red local.

### 3.1 Estructura

**Tabla 6**

| Carpeta | Contenido |
|---|---|
| `app/config` | i18next con `es`, `en`, `fr`, `pt` (`app/config/locales`) |
| `app/providers` | `AppProviders`, sincronización de las preferencias de la cuenta y del token de avisos push, y `AppUpdatesCheck` (si el servidor publicó un APK más nuevo, exige instalarlo antes de seguir) |
| `app/theme` | Contexto de tema, colores y tipografía |
| `core/api` | `config.ts` (URL del gateway y `WEB_URL` para los enlaces públicos), `httpClient.ts` (fetch con JWT, JSON o *multipart* y tiempo límite de 15 s o el que pida cada llamada), `apiError.ts` |
| `core/navigation` | `RootNavigator` (pantallas por rol) y tipos de rutas |

| Carpeta | Contenido |
|---|---|
| `core/services` | `auth`, `booking`, `vehicles`, `operations`, `payments` (con `receiptImage.ts`, que comprime el comprobante a máximo 2 MB), `notifications`, `users`, `appUpdates` (lee `/app/version.json`) |
| `core/storage` | Sesión en `expo-secure-store` (en web, AsyncStorage) |
| `modules/auth`, `client`, `operator`, `admin` | Pantallas, componentes, modelos, hooks y viewmodels de cada rol |
| `shared` | Layouts por rol, vistas compartidas (perfil, notificaciones, ajustes), validadores, componentes y utilidades |

## 3.2 Dependencias principales

### Tabla 7

*Librerías del móvil (`LavadoVehicular-Movil/package.json`)*

| Librería | Versión | Uso |
|---|---|---|
| `expo` | ~57.0.30 | Plataforma de compilación y ejecución |
| `react`, `react-native` | 19.2.3, 0.86.3 | Interfaz |
| `@react-navigation/native`, `native-stack` | ^7.3.13, ^7.18.5 | Navegación |
| `expo-secure-store`, `@react-native-async-storage/async-storage` | ~57.0.4, 2.2.0 | Guardar la sesión |
| `i18next`, `react-i18next` | ^26.4.1, ^17.0.13 | Traducciones |
| `expo-print`, `expo-sharing`, `expo-file-system`, `expo-document-picker` | ~57.0.x | Exportar reportes y adjuntar comprobantes |
| `expo-image-picker`, `expo-image-manipulator` | ~57.0.20, ~57.0.21 | Tomar o elegir las fotos del informe de inspección y reducirlas antes de subirlas |
| `expo-notifications`, `expo-updates` | ~57.0.23, ~57.0.26 | Notificaciones push y actualizaciones por internet (OTA) |
| `react-native-svg`, `react-native-reanimated` | 15.15.4, 4.5.1 | Gráficas y animaciones |

## 3.3 Configuración

La URL del backend se elige en este orden (`src/core/api/config.ts`):

1. `EXPO_PUBLIC_API_URL` en `.env.local`, si se define.
2. La IP del computador que corre Expo (la misma con la que el celular abrió la app).
3. Por plataforma: `http://10.0.2.2:8080/api/v1` en el emulador de Android y `http://localhost:8080/api/v1` en la versión web.

La plantilla `.env.example` explica las variables opcionales; `.env.local` no se sube al repositorio. El celular y el computador deben estar en la misma red wifi y el firewall debe permitir el puerto 8080.

### 3.4 Compilar y ejecutar

```
cd LavadoVehicular-Movil
npm install
npm start                # expo start: escanear el QR con Expo Go
npm run android          # abre en emulador o dispositivo Android
npm run web              # versión web en http://localhost:8081
npx expo start --clear   # reiniciar limpiando caché tras cambiar .env.local
```

Para desarrollo la app se ejecuta con Expo Go. El APK se compila con EAS (`eas.json`, perfiles `preview` y `production`), y los scripts `npm run publish:apk` y `npm run publish:update` publican un APK nuevo en el servidor o una actualización por internet (ver el Documento de Despliegue, carpeta 07).

## 4. Backend del que dependen los clientes

La web y el móvil solo necesitan el gateway en `localhost:8080`; detrás deben estar arriba los seis servicios, SQL Server y, si se activan los eventos, RabbitMQ. Todo se levanta desde `Backend/lavarapido-infra` con Docker.

```
cd Backend/lavarapido-infra
cp .env.example .env                          # completar secretos; el .env no se sube
docker compose up -d                          # solo SQL Server, Mailpit y RabbitMQ
(servicios desde el IDE)
docker compose --profile app up -d --build    # todo el backend en contenedores
```

**Tabla 8**

*Variables de `.env.example` (sin valores)*

| Grupo | Variables |
|---|---|
| Base de datos | `DB_PASSWORD` (obligatoria), `DB_NAME`, `DB_USERNAME`, `DB_URL` |
| Seguridad | `JWT_SECRET` (obligatoria), `INTERNAL_API_KEY` (llamadas entre servicios y asignación automática), `CORS_ALLOWED_ORIGINS` |
| Correo | `MAIL_ENABLED`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM` |

| Grupo | Variables |
|---|---|
| Mensajería | `MESSAGING_ENABLED`, `RABBITMQ_HOST`, `RABBITMQ_PORT`, `RABBITMQ_USERNAME`, `RABBITMQ_PASSWORD`; `AUTO_ASSIGN_ENABLED=false` (opcional) apaga solo la asignación automática |
| Push | `PUSH_ENABLED` (y `EXPO_ACCESS_TOKEN` en el compose) |

> *Nota.* Las variables del 21 de octubre de 2026 no están en `.env.example` ni en el compose porque tienen valor por defecto: `SESSION_CHECK_ENABLED`, `SESSION_CHECK_CACHE_TTL` y `SESSION_CHECK_TIMEOUT` (chequeo de sesión del gateway: activo, 30 s y 3 s); `BOOKING_EXPIRY_CHECK_ENABLED` y `BOOKING_EXPIRY_CHECK_INTERVAL` (cancelación de las reservas programadas que vencieron sin confirmarse, en booking-service: activa y cada 5 minutos); `BOOKING_EXPIRY_GRACE_MINUTES` (60 minutos de margen después del fin de la reserva, que usan booking-service y operations-service), y `FOLLOW_UP_ENABLED` y `FOLLOW_UP_INTERVAL` (seguimiento de las reservas que terminaron y nadie cerró, en operations-service: activo y cada 5 minutos; necesita `INTERNAL_API_KEY`). Solo cambian si el servicio se corre desde el IDE o si se agregan al `environment` del servicio en el compose.

Para correr un servicio Java desde la consola se usa su Maven Wrapper (`./mvnw spring-boot:run` dentro de la carpeta del servicio) con Java 21 (Spring, s.f.). payment-service se ejecuta con `dotnet run --project src/PaymentService.Api` y requiere el SDK de .NET 8 (Microsoft, s.f.). Al levantar con `--profile app`, las carpetas deben llamarse como sus repositorios (`lavarapido-security-service`, `lavarapido-operation-service`), porque así las referencia el compose.

**Tabla 9**

*Puertos para verificar que todo está arriba*

| Servicio | URL de verificación |
|---|---|
| API Gateway | `http://localhost:8080/actuator/health` |
| Servicios Java | `http://localhost:300X/actuator/health` (X = 1, 2, 3, 4, 6) y Swagger en `/swagger-ui.html` |
| payment-service | `http://localhost:3005/actuator/health` y `/health` (los dos revisan SQL Server y responden 503 si no contesta); Swagger en `/swagger` solo en entorno Development (`Program.cs:53-57`) |
| RabbitMQ | Panel en `http://localhost:15672` |
| Mailpit | Bandeja en `http://localhost:8025` |

Con todo arriba, `lavarapido-infra/e2e` tiene tres tipos de scripts de Node 22: las 19 pruebas de punta a punta (`node --test "e2e/*.test.js"`: flujo de reserva entre servicios, flujo completo de los tres roles, informe de inspección y reglas de acceso), la medición de tiempos de respuesta (`node e2e/performance.js`) y la revisión de la web a 360 px de ancho (`node e2e/width-360.js`, después de `npm install --prefix e2e`). Necesitan `MESSAGING_ENABLED=true` e `INTERNAL_API_KEY` en el `.env`.

## Referencias

Angular. (s.f.). *Angular documentation*. Google. Recuperado el 9 de octubre de 2026, de https://angular.dev/

Expo. (s.f.). *Expo documentation*. Recuperado el 9 de octubre de 2026, de https://docs.expo.dev/

Microsoft. (s.f.). *.NET documentation*. Recuperado el 9 de octubre de 2026, de https://learn.microsoft.com/dotnet/

React Native. (s.f.). *React Native documentation*. Meta Platforms. Recuperado el 9 de octubre de 2026, de https://reactnative.dev/docs/getting-started

Spring. (s.f.). *Spring Boot reference documentation*. Broadcom. Recuperado el 9 de octubre de 2026, de https://docs.spring.io/spring-boot/